

AI-Based Fake Identity & Document Screening

Testing + Demo Quick-Reference | SSB Hackathon | 8-9 day timeline

1. Testing Order — Bottom-Up

Never start with the full pipeline. Test each module in isolation first, then wire up.

#	Module	How to test
1	Module 1 - OCR	Call <code>run_ocr()</code> directly on clean + 3 tampered fixtures. Compare <code>extracted_fields</code> vs the <code>.json</code> manifests.
2	Module 2 - Validation	Feed it REAL Module 1 output (not hand-written mocks). Check <code>tampered_02</code> gives <code>text_mrz_mismatch_dob</code> .
3	Module 3 - Tampering	Run directly on image bytes. Compare <code>tamper_verdict</code> vs <code>expected_module3_result</code> in each JSON. VERIFY, don't assume — see note below.
4	Module 4 - Face	Run on 5 real (doc, selfie) pairs -> match. Run cross-pairs (Ashray doc + Riya selfie) -> mismatch. Synthetic passports (no real photo) -> <code>no_face_detected</code> .
5	Module 5 - Authority	Once <code>mock_db2</code> filled: same doc+number -> match. Different person's doc under someone else's number -> mismatch.
6	Module 6 - Ledger	Call <code>write_ledger_record()</code> twice: same face + same identity (no flag), then same face + new fabricated ID (<code>identity_reuse_flag=True</code>).
7	Risk Engine	Hand-construct edge-case dicts (e.g. <code>authority=mismatch</code> + everything else clean) to prove the FLOOR overrides the weighted average.
8	Full Pipeline	FastAPI TestClient hitting <code>/screen</code> with the same fixtures. Confirm final JSON is coherent end to end.

■ Correction to remember: the DOB-alter fixture's **expected_module3_result** is a PREDICTION your team wrote, not a guarantee. ManTraNet/ELA may or may not fire depending on exactly how the text region was edited/resaved. Actually run it and see — then narrate whatever the real output is in the demo.

2. Data Map — What Each Fixture Is FOR

Fixture set	Use for	Do NOT use for
<code>clean_passport_01</code> + 3 tampered variants	Module 1, 2, 3 only	No real face -> never use for Module 4/5
MIDV / Greek passport (<code>CA25_01</code>)	Module 1 OCR generalization only	Not tied to mock DB or any selfie — don't force it into face/authority tests
5 teammates' document + selfie pairs	Module 4, 5, 6b, and full <code>/screen</code> runs	Your ONLY real face data — use for every face-related and end-to-end scenario

Known gap: `mock_db2_authority_ref.py` needs entries added for F1000001-F1000005 -> each teammate's own document photo. No new photos needed anywhere else.

3. Building Every Scenario From Existing Data (zero new photos)

Scenario	How to construct it	Expected result
Clean / match	Teammate's own doc + own selfie, correct passport number	clear, high evidence
MRZ/text tamper	<code>tampered_02_dob_alter</code>	Module 2 flags <code>text_mrz_mismatch_dob</code>
Copy-move tamper	<code>tampered_03_stamp_clone</code>	Module 3 flags <code>copy_move_flag</code>

Scenario	How to construct it	Expected result
Face mismatch	Teammate A's document + Teammate B's selfie, same declared number	Module 4 mismatch
Authority mismatch (flawless forgery)	Teammate B's document photo declared under Teammate A's real passport number	Module 5 mismatch -> risk floor forces high_risk
Identity reuse	Same teammate's doc+selfie run twice: 1st under real number, 2nd under a FABRICATED never-issued number	2nd call: identity_reuse_flag=True

4. Pre-Demo Testing Checklist

- [] Module 1 OCR run on all 4 passport fixtures + MIDV image — fields extracted, confidences present
- [] Module 2 validation run on real Module 1 outputs — checksum, text/MRZ crosscheck, DB lookup all verified
- [] Module 3 tampering run on clean + 3 tampered — actual tamper_verdict recorded (not assumed)
- [] Module 4 face — 5 match pairs + at least 1 cross-pair mismatch run
- [] mock_db2_authority_ref.py populated with F1000001–F1000005
- [] Module 5 authority — 1 match case + 1 mismatch case run
- [] Module 6a ledger — hash chain verified (tamper one record, confirm chain breaks)
- [] Module 6b identity reuse — same face/new fabricated ID triggers flag; same face/same ID does not
- [] Risk engine — floor logic manually confirmed to override weighted average
- [] Full /screen pipeline run via FastAPI TestClient for all 6 scenarios in Section 3
- [] /ledger endpoint returns full chain incl. persisted authority_match_status + identity_reuse fields
- [] requirements-lock.txt environment double-checked on a clean install before demo day

5. Minimal Frontend Needed

Backend test scripts are for YOUR validation, not for judges. Judges need to watch: upload doc -> upload selfie -> see reasoning -> see verdict. Build one thin page, not a polished product:

- Single page: doc-type dropdown + 2 file uploads (document, selfie) + Screen button
- Result card: risk_band color-coded (green/yellow/red), evidence_strength, contributing_flags list
- Collapsible raw JSON section for technical judges (OCR fields, tamper scores, etc.)
- Second tab: Ledger/Audit Trail view hitting GET /ledger — this visually sells Module 6

6. Live Demo Script — Escalating Story, Not Random Runs

Step	Action	Talking point
1	Clean, real teammate doc+selfie	clear, high evidence
2	tampered_02_dob_alter	Module 2 catches text/MRZ mismatch
3	tampered_03_stamp_clone	Module 3 catches copy-move clone
4	Cross-pair face mismatch	Module 4 flags mismatch
5	Authority mismatch (flawless forgery)	Say it explicitly: 'everything else is clean — only Module 5 catches this'
6	Identity reuse (fabricated new ID)	Show ledger tab -> identity_reuse_flag=true, explain cross-checkpoint reasoning

7. Key Talking Points To Remember

- "No evidence of fraud ≠ evidence of no fraud" — clear + low evidence downgrades to secondary_check, never silently passes.
- Module 5 catches a FORGED version of a real identity. Module 6b catches a forged identity that NEVER EXISTED at all.
- Floor logic, not pure averaging: a confident Module 5/6 signal can never be mathematically outvoted by 3 clean signals.

- Module 3 must never use `correct_geometry()/normalize_contrast()` — resampling corrupts ELA's forensic signal.
- Name the prototype-vs-real-world gaps OUT LOUD: mock DB stands in for RFID/chip read; uploaded selfie stands in for live camera capture. This reads as engineering maturity, not a weakness.
- Blockchain layer persists `authority_match_status` and `identity_reuse` flags PERMANENTLY into the ledger — not just returned once and discarded.

Reference: `action_plan.pdf` Sections 4, 5, 7, 9a/9b, 11, 12 | `schema/module_io_schema.json` is the authoritative I/O contract